

Flutter Expert Interview Questions

Part 3: Localization • Accessibility • Package Publishing • Advanced Patterns

Questions 66-115: Expert-level topics for architects and tech leads

Table of Contents

1. Internationalization & Localization
2. Accessibility (a11y)
3. Package Development & Publishing
4. Advanced State Management Patterns
5. Navigation & Deep Linking
6. Device Features & Permissions
7. Custom Widgets & Rendering
8. Advanced Dart & Async Patterns
9. Code Generation & Build Runners
10. Debugging & Monitoring

1. Internationalization & Localization

Q66. What is i18n/l10n in Flutter and how do you implement it?

i18n (Internationalization): Building apps to support multiple languages/locales.

l10n (Localization): Adapting content for specific regions.

Implementation:

1. intl package: Dart package for i18n
2. Set up: Create ARB files for translations
3. Code generation: flutter gen-l10n generates classes
4. MaterialApp: Configure localizationsDelegates
5. Usage: AppLocalizations.of(context)
6. DateTime/Number formatting: Use intl for locale-specific formatting

Q67. Explain ARB (Application Resource Bundle) file format.

ARB format: JSON-based format for managing translations. Files named app_en.arb, app_hi.arb, etc.

Features: Placeholders for variables, descriptions for translators, pluralization support, multiple locales.

Benefits: Single source of truth, easy maintenance, translator-friendly, type-safe code generation.

Q68. How do you format dates, numbers, and currencies for different locales?

Using intl package: DateFormat for dates, NumberFormat for numbers, currency formatting.

For Trackk (India): NumberFormat.currency(locale: 'en_IN', symbol: '₹').format(amount)

Always format based on locale, never hardcode formats. This handles proper symbols and spacing for Indian rupee.

Q69. How do you handle RTL (Right-to-Left) languages in Flutter?

RTL support: Flutter detects RTL locales (Arabic, Hebrew, Persian) automatically.

Best practice: Use semantic properties (start/end) instead of directional (left/right).

Use EdgeInsets.only(start, end), Alignment.topStart, and let Flutter handle automatic reordering for RTL.

Q70. How do you implement a language switcher in your app?

Language switcher: Store selected locale in provider/BLoC, persist to SharedPreferences.

Update MaterialApp's locale parameter with selectedLocale.

Use Provider to notify listeners and rebuild app when language changes (no restart needed).

2. Accessibility (a11y)

Q71. What is accessibility and why is it important in Flutter?

Accessibility: Ensures apps are usable by everyone including people with disabilities.
Covers visual, hearing, motor, and cognitive disabilities.
Benefits: Larger user base, legal compliance (WCAG), better UX for all.

Q72. How do you implement semantic labels and screen reader support?

Screen reader implementation: Use Semantics widget for context, provide meaningful labels for buttons, add semantic labels to images.
Use `Image.asset('icon.png', semanticLabel: 'Home icon')`.
Test with TalkBack (Android) and VoiceOver (iOS).

Q73. How do you handle focus and keyboard navigation?

Focus management: Use FocusNode for programmatic control, FocusScope for groups.
Set `tabIndex` or `autofocus: true` for tab order.
Ensure logical navigation flow for keyboard users.

Q74. What are the WCAG accessibility standards and how do you ensure compliance?

WCAG 2.1 Standards: Four principles: Perceivable, Operable, Understandable, Robust.
Compliance checklist: Minimum 4.5:1 color contrast, semantic labels, keyboard navigation, 48x48 dp touch targets, no time-based interactions without controls.

Q75. How do you test accessibility in Flutter apps?

Testing approach: Manual testing with TalkBack/VoiceOver, automated tests with `semanticsEnabled`.
Use Flutter DevTools Accessibility panel, real devices, and include people with disabilities in testing.

3. Package Development & Publishing

Q76. What is the difference between Dart packages and Flutter plugins?

Dart Package: Pure Dart code, no native code, works on any platform.

Flutter Plugin: Includes native code (Kotlin, Swift), bridges Dart and native.

Examples: json_serializable (package), camera/geolocator (plugins).

Your competitive advantage: Published Dart package on pub.dev

Q77. How do you create and publish a Dart package to pub.dev?

Steps: Create package structure, configure pubspec.yaml, write tests, get pub.dev account, run 'flutter pub publish'.

Structure: lib/ (public API), test/ (unit tests), README.md, CHANGELOG.md, LICENSE.

Use semantic versioning (major.minor.patch).

Q78. What should be included in a well-documented Dart package?

Documentation: README.md with overview/features/usage, CHANGELOG.md with version history, CONTRIBUTING.md, example app, API documentation with Dartdoc comments, LICENSE, test coverage, badges.

Q79. How do you handle versioning and breaking changes in packages?

Semantic Versioning: MAJOR.MINOR.PATCH

Breaking changes: Bump MAJOR version, document clearly in CHANGELOG, provide migration guide.

Non-breaking features: Bump MINOR. Bug fixes: Bump PATCH.

Q80. What is pubspec.yaml and how do you manage dependencies?

pubspec.yaml: Defines package metadata, dependencies (runtime), dev_dependencies.

Version constraints: ^1.2.3 (caret), ~1.2.3 (tilde), explicit ranges.

Best practice: Use caret for stable versions, lock in dev for reproducibility.

4. Advanced State Management Patterns

Q81. Explain the Repository Pattern and Data Mapper Pattern.

Repository Pattern: Abstract data access logic from business logic.

Data Mapper: Maps domain entities to data models.

Benefits: Easy to swap implementations, testable, centralized data access. Essential for clean architecture.

Q82. What is CQRS and when to use it?

CQRS: Separate read and write operations into different models.

Benefits: Independent optimization, scalability, clarity.

Use cases: Complex reads, performance-critical apps. Avoid for simple apps.

Q83. Explain the MVVM pattern and how it differs from MVC.

MVVM: View binds to ViewModel (like BLoC in Flutter), cleaner separation.

Advantages: Two-way binding, testable ViewModel, reusable across views.

Q84. What is Event Sourcing?

Event Sourcing: Store state as sequence of events, rebuild by replaying events.

Benefits: Complete audit trail, debugging via replay, time travel.

Essential for financial apps requiring audit logs (important for Trackk).

Q85. How do you implement global state management with Provider?

Provider pattern: Create ChangeNotifier class, provide globally with MultiProvider.

Consume using Consumer or context.watch()/read().

Use Selector for rebuilding on specific property changes.

5. Navigation & Deep Linking

Q86. What is deep linking and how do you implement it?

Deep linking: Navigate directly to specific screen via URL.

Use cases: Push notifications, email links, sharing.

Implementation: Configure intent filters (Android), URL schemes (iOS), use `app_links` package to parse and navigate.

Q87. Explain named routes vs route generators.

Named routes: Pre-defined routes, simple but limited.

Route generators: Programmatic with parameter parsing, more flexible.

Route generator is preferred for real-world apps.

Q88. How do you implement Navigator 2.0 with GoRouter?

GoRouter: Simplifies Navigator 2.0 with declarative routing.

Benefits: Type-safe, automatic deep linking, state restoration.

Use `context.go()` or `context.push()` for navigation.

Q89. How do you handle the back button and navigation state?

Back button handling: Use `PopScope` for modern handling, check `Navigator.canPop()`.

Prevent exit with confirmation dialogs for unsaved data.

Q90. How do you manage navigation state across orientation changes?

State preservation: Use `RestorationId`, `GoRouter` auto-restores state.

Persist route stack to `SharedPreferences` if needed.

Verify state restored after configuration changes.

6. Device Features & Permissions

Q91. How do you handle permissions in Flutter?

permission_handler package: Cross-platform solution.
Check with `Permission.camera.status`, request with `Permission.camera.request()`.
Setup: Add keys to `Info.plist` (iOS), `AndroidManifest.xml` (Android).
Best practice: Request only when needed.

Q92. How do you implement camera functionality?

camera package: Initialize `CameraController`, `CameraPreview` widget, `takePicture()`.
Always dispose controller and request permissions first.

Q93. How do you implement location services and GPS tracking?

geolocator package: Request location permission, `getCurrentPosition()`, listen to position streams.
Distance calculation: `Geolocator.distanceBetween()`.

Q94. How do you implement push notifications?

firebase_messaging package: Configure Firebase, get FCM token, handle messages (foreground/background).
Deep linking: Include route in notification payload.
AppsFlyer integration: Save token for uninstall tracking (Trackk use case).

Q95. How do you access device contacts and calendar?

Contacts: `fast_contacts`, `ContactsService` packages.
Calendar: `device_calendar` package for events.
Privacy: Handle carefully, never store without permission.

7. Custom Widgets & Rendering

Q96. How do you create custom widgets and when to use CustomPaint?

Custom widgets: Compose existing widgets (preferred).

CustomPaint: For drawing graphics not achievable with widgets.

Use for: Charts, graphs, custom shapes, real-time drawing.

Q97. Explain RenderObject and custom render objects.

RenderObject: Low-level rendering layer, handles layout and painting.

Create custom RenderObject for: Complex layouts, custom geometry, performance-critical code.

Advanced topic: Use carefully due to complexity.

Q98. How do you optimize rendering and reduce repaints?

Optimization: RepaintBoundary for expensive widgets, const constructors, proper shouldRepaint() implementation.

Profile with DevTools performance tab to visualize repaints.

Q99. How do you implement theming in Flutter?

Theming: ThemeData in MaterialApp, Theme.of(context) to access values.

Multiple themes: Store selection in provider, rebuild MaterialApp.

Dark mode: Use ThemeMode.dark and darkTheme property.

8. Advanced Dart & Async Patterns

Q100. Explain async/await and how they work.

async/await: Syntactic sugar for Futures, makes async code readable.
async: Function always returns Future.
await: Pauses execution until Future completes.
Cleaner than `.then()` chaining.

Q101. What is the difference between Stream, Future, and StreamSubscription?

Future: Single async value, completes once, use for API calls.
Stream: Multiple values over time, stays open, use for real-time data.
StreamSubscription: Connection to stream, must cancel to avoid leaks.

Q102. How do you handle errors in streams and futures?

Futures: try-catch with `async/await`, `.catchError()`.
Streams: `onError` callback in `listen()`, `StreamBuilder` error handling.

Q103. What are generators and how do you use them?

Sync generator (sync*): Produces Iterable.
Async generator (async*): Produces Stream.
Use for: Lazy evaluation, pagination, streaming data from API.

Q104. How do you implement cancellation and timeouts?

Timeout: `await future.timeout(Duration(seconds: 30))`.
CancelToken (dio): For cancelling HTTP requests.
Use case: Cancel when user navigates away.

9. Code Generation & Build Runners

Q105. What is code generation and why is it important?

Code generation: Automatic code creation from annotations.
Benefits: Boilerplate reduction, type safety, consistency.
Tools: build_runner, build, analyzer.

Q106. How do you use json_serializable?

json_serializable: Auto-generate fromJson/toJson methods.
Annotate class with @JsonSerializable(), run build_runner.
Supports custom converters and field renaming with @JsonKey.

Q107. How do you use freezed for immutable classes?

freezed: Generate immutable models with copyWith().
Auto-generates: equality, hashCode, toString(), pattern matching.
Benefits: Immutability, easier state management, less boilerplate.

Q108. How do you optimize build runner performance?

Optimization: Use watch mode, clean rebuild if issues, delete .dart_tool/.
Split packages for faster builds, monitor with --verbose flag.

10. Debugging & Monitoring

Q109. How do you implement comprehensive logging?

logger package: Structured logging with levels (Verbose, Debug, Info, Warning, Error).
Remote logging: Sentry, Firebase Crashlytics.
Best practice: Don't log sensitive data.

Q110. How do you use Sentry/Firebase Crashlytics?

Firebase Crashlytics: Logs crashes, custom messages, exceptions.
Sentry: Error tracking with breadcrumbs and device info.
Both track: Crashes, exceptions, device info, app version, user actions.

Q111. How do you debug memory leaks?

Common causes: Unreleased streams, undisposed controllers, uncanceled timers, listeners not removed.
Prevention: Always dispose in dispose(), remove listeners, cancel streams.
Debug: Use DevTools Memory tab to take snapshots.

Q112. How do you implement feature flags and A/B testing?

Firebase RemoteConfig: Define flags in Console, fetch and activate.
A/B testing: Assign users to variants, track conversions.
Use cases: Gradual rollout, testing UI changes, performance experiments.

Bonus: Expert Scenario Questions

Q113. Describe your approach to designing a large-scale trading platform (Trackk scenario).

Architecture: Clean architecture with BLoC/Riverpod.

Data layer: REST APIs, WebSocket for real-time prices, offline support.

Real-time: WebSocket updates, push notifications, price alerts.

Security: Certificate pinning, token refresh, encrypted storage.

Performance: Efficient lists, image caching.

Monitoring: Firebase, Sentry, custom analytics.

CI/CD: Automated testing and deployment.

Q114. How would you handle migration from Provider to Riverpod?

Gradual migration: Feature by feature, not all at once.

Coexistence: Both can work together temporarily.

Testing: Comprehensive tests before/after.

Timeline: Realistic schedule with buffer for issues.

Communication: Update team on new patterns.

Q115. What metrics would you track in a production app?

Performance: Frame time, startup time, memory, battery.

Stability: Crash rate, ANR rate, error rate.

User experience: Session length, feature adoption, retention.

Code quality: Test coverage (>80%), lint violations.

Business: Conversion rates, API response times.

Tools: Firebase Analytics, Crashlytics, Sentry.